GépészBüfé

Tesztelői Dokumentáció

Iskolai büfé online rendelési rendszer

Kiss Imre, Kovács Attila, Hevesi Tamás
Mátészalka, 2026. április 15.

Tartalomjegyzék

1. A tesztelés célja és hatóköre 3
2. Tesztkörnyezet és hardver-szoftver architektúra 3
2.1. Hardveres környezet 3
2.2. Szoftveres és technológiai környezet 4
3. Alkalmazott tesztelési stratégiák 4
4. Funkcionális és Moduláris Tesztesetek 5
4.1. Felhasználói hitelesítés (Auth) 5
4.2. Étlap, Kosár és Keresés 5
4.3. Rendelés leadása és Valós idejű követés 6
4.4. Adminisztrációs Irányítópult 6
5. UI/UX és Reszponzivitás Ellenőrzése 7
6. Biztonsági és Hozzáférési Tesztek 7
7. Teljesítmény és Terhelési Mutatók 8
8. Hibajegyzék és Javítási Napló (Bug Tracker) 8
9. Felhasznált Fejlesztői és Tesztelési Eszközök (MERN Stack) 9
10. Záró Értékelés és Következő Fejlesztési Irányok 10

1. A tesztelés célja és hatóköre

A jelen dokumentum celja a GépészBüfé nevű webes alkalmazás (MERN stack alapokon nyugvó rendszer) teljes körű minőségbiztosítási és tesztelesi folyamatának dokumentálása. A tesztelés során a célunk az volt, hogy validáljuk a diákok és a büfé üzemeltetői számára fejlesztett funkciókat, garantáljuk a rendelések zökkenőmentes rögzítését, a stabil adatbázis-kapcsolatot (MongoDB), a gyors frontend működést (React), illetőleg a biztos lábakon álló szerveroldali logikát (Node.js/Express).

2. Tesztkörnyezet és hardver-szoftver architektúra

2.1. Hardveres környezet

A tesztelési folyamat az alábbi hardverkonfigurációkon és eszközökön zajlott:

- Asztali gépek: Fejlesztői PC (Windows 11, min. 16 GB RAM)
- Hordozható gépek: Szabványos laptopok, DELL, ASUS, MCBOOK
- Mobileszközök: Android (Samsung A32, Xiaomi), iOS (iPhone 13, 14 platformok)

2.2. Szoftveres és technológiai környezet

Az alábbiakban felsoroljuk a tesztelési folyamathoz igénybe vett szoftvereket és technológiákat:

- Böngészők: Google Chrome (120+), Mozilla Firefox, Microsoft Edge, Safari iOS-en.
- Frontend / Kliens oldal: React.js, Vite, HTML5, CSS3.
- Backend / Szerver: Node.js, Express.js.
- Adatbázis: MongoDB (MongoDB Atlas / localhost).
- Valós idejű kapcsolat: Socket.io (rendelések élő követéséhez).
- Fejlesztői környezet: Visual Studio Code.

Tesztelési környezetek összefoglalója

Környezet	Konfiguráció
Windows 11 + Chrome 122	1920×1080 px, full HD – elsődleges fejlesztői gép
Windows 11 + Firefox 124	1920×1080 px – keresztböngésző teszt
Android 14 + Chrome Mobile	Samsung Galaxy A32, 1080×2340 px – mobilos megjelenítés
iOS 17 + Safari	iPhone 13, 2532×1170 px – Apple mobilos ellenőrzés
Windows 11 + Edge 122	1920×1080 px – Microsoft böngésző kompatibilitás
Kis képernyő szimulált	DevTools 375×812 px (iPhone SE) – reszponzivitás

Megállapítások: Az alkalmazás minden tesztelt böngészőben és eszközön helyesen jelent meg és működött. A mobilos nézetben a Navbar hamburger menüre vált, a kosár és a termékrács is reszponzív

egyhasábos elrendezésbe rendeződik. Az adminisztrációs panelen is minden rendben működött mobil nézetben.

3. Alkalmazott tesztelési stratégiák

A GépészBüfé rendszer ellenőrzése során az alábbi vizsgáló eljárásokat vettük igénybe:

1. Egységtesztelés (Manual Unit Testing): Az API végpontok önálló, Postman szoftverrel történő szimulációja és tesztje (függetlenül a kliens oldaltól).
2. Integrációs Tesztelés: A frontend állapottérének (React Context) és a backend adatbázis (MongoDB) kommunikációjának összehangolt működésének vizsgálata.
3. End-to-End (E2E) Teszt: A teljes folyamat eljátszása a diák bejelentkezésétől, az egyedi kosár összeállításán át, egészen az étel átvételéig és a státusz admin felületi frissítéséig.
4. Reszponzivitás vizsgálat: Dinamikus nézetellenőrzés mobil, tablet és monitor méreteken.
5. Regressziós teszt (Visszalépési teszt): Korábban kijavított hibák újraellenőrzése a funkcióbővítések után.

Szemben a hagyományos (pl. tisztán PHP alapú) rendszerekkel, itt a frontend és a backend szét van választva, így a hibakeresés két pilléren futott: a React Developer Tools-szal a kliens oldali állapottér valós idejű ellenőrzése, Postmannel pedig az Express.js API végpontok önálló tesztelése volt elvégezhető.

4. Funkcionális és Moduláris Tesztesetek

4.1. Felhasználói hitelesítés (Auth)

ID	Tesztelt Funkció	Lépések	Elvárt Eredmény	Státusz
AUTH-01	Regisztráció új diákként	1. Pop-up megnyitása 2. Érvényes adatok beírása 3. Küldés	Fiók létrejön, JWT token tárolódik, user bejelentkezik.	✅Sikeres
AUTH-02	Hibás Regisztráció (Már létező email)	1. Célzottan létező email megadása 2. Küldés	Toast error: "Ez az email cím már foglalt."	✅Sikeres
AUTH-03	Bejelentkezés	1. Helyes adatok bevitele 2. Gombnyomás	Hozzáférés a kosárhoz, Profil menü megjelenik a navigációban.	✅Sikeres
AUTH-04	Kijelentkezés	1. Profil -> Kijelentkezés klikk	JWT törlődik, átirányítás a főoldalra, kosár ürül.	✅Sikeres

4.2. Étlap, Kosár és Keresés

ID	Tesztelt Funkció	Lépések	Elvárt Eredmény	Státusz
MENU-01	Termék kosárba tétele	1. '+' ikon egy ételre	A kosár jelvény (dot) megjelenik, az étel számlálója 1-re ugrik.	✅Sikeres
MENU-02	Mennyiség növelése/csökkenése	1. '-' gomb nyomása 1-es értéknél	A termék kikerül a kosárból.	✅Sikeres
MENU-03	Kategória szűrés (Pl.: Szendvicsek)	1. Felső Explorer menü -> "Szendvics"	Csak a szendvics kategóriájú termékek listázódnak.	✅Sikeres
MENU-04	Termékkereső működése	1. Navbar kereső megnyitása 2. "Kóla" beírása	A megjelenített ételek szűkülnek a beírt kulcsszó alapján.	✅Sikeres

4.3. Rendelés leadása és Valós idejű követés

ID	Tesztelt Funkció	Lépések	Elvárt Eredmény	Státusz
ORD-01	Sikeres vásárlás leadása	1. Pénztár oldal 2. Kötelező adatok 3. "Megrendelés"	Sikeres rendelés, véletlenszerű kóddal (pl. #A5B6), ugrás a success oldalra.	✅Sikeres
ORD-02	Websocket státusz frissítés (Élő)	1. Diák a "Rendeléseim" oldalon 2. Admin módosítja a státuszt "Elkészült"-re.	Az oldal frissítése nélkül a diák zöld Toastify felugrót kap: "Elkészült a rendelésed!".	✅Sikeres
ORD-03	Kosár validáció (Üres kosár ütközés)	1. Kosár üres 2. "/order" URL direkt megnyitása	A rendszer visszadob a "/cart" vagy főoldalra.	✅Sikeres

4.4. Adminisztrációs Irányítópult

ID	Tesztelt Funkció	Lépések	Elvárt Eredmény	Státusz
ADM-01	Admin oldal betöltése	1. localhost:5174 2. Helyes/Belépett állapot	Stabilitás, oldalsáv betöltődik.	✅Sikeres
ADM-02	Új menüelem feltöltése	1. "Hozzáadás" lap 2. Kép, Név, Ár, Kategória felvitele 3. Mentés	Az étel bekerül a MongoDB foods kollekcióba, kép bekerül a backend uploads mappába.	✅Sikeres
ADM-03	Listázott ételek törlése	1. "Lista" oldal 2. Kuka ("X") ikon az étel mellett.	Törlődik az adatbázisból és az uploads mappából a fájl is.	✅Sikeres

ADM-04	Aktív rendelések listázása	1. "Rendelések" fül	A diákok által leadott rendelések blokkjai legkorábbitól datálva megjelennek.	✅Sikeres

5. UI/UX és Reszponzivitás Ellenőrzése

Célunk az volt, hogy a büfé alkalmazása tableten szünetben és mobiltelefonon a folyosón is 100%-os élményt nyújtson a tanulóknak.

Modul vagy Nézet	Asztali (1080p+)	Tablet (768px – 1024px)	Mobil (< 768px)	Minősítés
Navigáció (Navbar)	Teljes menü, ikonok jobbra, kereső nyitott.	Ikonok maradnak, menüpontok középen.	Hamburger menü, felugró Sidebar, ikonsáv tömörítve.	□ Kiváló
Étel Grid Kártyák	4–5 oszlop	2–3 oszlop	1 oszlop, széles kártyákkal	□ Kiváló
Kosár / Pénztár	Két oszlop (Tételek balra, Összegzés jobbra)	Két oszlop	Egymás alá rendezve (Flex-direction: column)	□ Kiváló
Sötét / Világos mód (Dark Mode)	Zökkenőmentes CSS váltás a var() változókkal.	Hibátlan kontraszt.	Tökéletesen olvasható OLED és normál mobilkijelzőkön is.	□ Kiváló

6. Biztonsági és Hozzáférési Tesztek

A GépészBüfé esetében – mivel pénzügyi/személyes tranzakciók (név, hely, diákigazolvány szám esetleg) utazhatnak az éteren – a MERN stack védelmi mechanizmusait be kellett mérni.

JWT (JSON Web Token) Teszt:

- Jelenlét: A bejelentkezést követően a backend csak valid tokennel engedi az authMiddleware.js rétegen át a "Rendeléseim" oldalt letölteni.
- Lejárt Token teszt: Tesztelve egy manuálisan lejáratott tokennel -> Az API 401 Unauthorized hibát dob, kliens kijelentkezteti a diákot.

XSS (Cross-Site Scripting) megelőzés:

A React a JSX miatt beépített védelemmel rendelkezik a kódbeszúrás ellen (automata escape). Tesztelésünk során megerősítettük, hogy szkript-injekciós kísérletek minden esetben sértetlenül string-ként kerültek megjelenítésre.

CORS beállítások:

Az backend API (server.js) úgy lett konfigurálva, hogy csak a saját frontend originjeiről (pl. localhost:5173, localhost:5174, vagy az éles domain) fogadjon POST/GET kéréseket. Ezt Postmanből és egyéb originen futó tesztkliensből is leellenőriztük.

Jelszóvédelem:

Az adatbázisban a diákok és adminok jelszavai bcrypt modullal titkosítva, sózva (salt) tárolódnak. MongoDB Compassszal ellenőriztük, hogy egyetlen mentett dokumentumban sem szerepel plain text jelszó.

7. Teljesítmény és Terhelési Mutatók

Entitás	Eredmény / Válaszidő	Megjegyzés
Képek Betöltése	~ 20 – 50 ms	Statikus route-on keresztül Node-ból served (/images) gyors.
API Válasz: /api/food/list	~ 80 ms	MongoDB lekérdezés, minimális indexeléssel is hatékony.
Kosár renderelés	< 15 ms	A React ContextAPI biztosítja a késleltetés nélküli, lokális update-et.
Alkalmazás mérete (Lighthouse)	Performance pontszám: 92/100	Jól optimalizált Vite build (npm run build).

Postman Collection Runner segítségével 100 párhuzamos rendelésleadási kérelmet küldtünk a szervernek. Az összes kérés 2,3 másodpercen belül feldolgozásra került, az API átlagos válaszideje 187 ms volt. Kódütközés (azonos randomCode két rendelésen) a tesztelés során egyszer sem fordult elő, ami a 4 jegyű kód 9000-es szimultán kapacitásának elégségességét igazolja.

8. Hibajegyzék és Javítási Napló (Bug Tracker)

Ez a szakasz a fejlesztés során észlelt, majd később javított anomáliákat tartalmazza.

ID	Hiba leírása (Súlyosság)	Ok / Gyökérelemzés	Alkalmazott Javítás	Státusz
B-01	Magas: A kosár kiürült oldalfrissítéskor (F5).	A ContextAPI állapot csak memóriában élt, nem perzisztált le.	A kosár aszinkron mentést kapott a böngésző localStorage-ébe / token alapú adatbázis mentéssel.	**JAVÍTVA**
B-02	Kritikus: Mobil nézetben csúszott a sárga Gépész logó, belement a hamburger menübe.	A CSS flex-wrap hiánya és a fix width érték okozta ütközés.	Média lekérdezés (@media max-width: 768px) bővítése, rugalmas ikon átméretezés.	**JAVÍTVA**
B-03	Közepes: Éles weben az admin képfeltöltés nem mentett fájlt (Cross-OS útvonal hiba).	A Node.js multer a backslash (\) helyett perjelet (/) várt Linux alatt.	Kijavítva a path.join(__dirname) natív JS fájlútvonal generátor használatával.	**JAVÍTVA**
B-04	Alacsony: A Socket.io többször dobott ki Toastify notification-t.	A useEffect hooks a Reactben kétszer futott le (Strict Mode), így több azonosító regisztrált.	A socket.disconnect() hozzáadása a UseEffect takarító (cleanup) funkciójához (return () => ...).	**JAVÍTVA**

9. Felhasznált Fejlesztői és Tesztelési Eszközök (MERN Stack)

Szemben a hagyományos (pl. tisztán PHP alapú) rendszerekkel, itt a frontend és a backend szét van választva, így a hibakeresés két pilléren futott:

6. React Developer Tools (Böngésző kiegészítő): Használata: A StoreContext (alkalmazásszintű memória) tartalmának valós idejű lekérdezése, az összetevők prop-jainak ellenőrzése.
7. Postman (API kliens): Használata: Az express router (pl. userRoute.js, foodRoute.js) végpontjainak tesztelése frontend nélkül. Kérések szimulációja POST/GET metodikával JSON küldéssel.
8. MongoDB Compass: Használata: Grafikus asztali kliens az adatbázis dokumentumainak böngészésére, a mentett rendelések közvetlen adatbázis szintű változtatására.
9. Chrome DevTools (Console, Network fül): Használata: API hálózati kommunikáció és visszatérési idők, illetve CORS hibák naplózása.
10. Nodemon: Használata: Backend fejlesztői szerver, ami azonnal újraindult bármilyen tesztelési módosítás után időmegtakarítást nyújtva.

10. Záró Értékelés és Következő Fejlesztési Irányok

Összesítő kijelentés:

A tesztelési fázis lezárultával kimondható, hogy a GépészBüfé projekt egy rendkívül gyors, funkciójában gazdag és masszív MERN architektúrával bíró e-commerce rendszer. Az éles bevetésre teljes mértékben alkalmas. A hibák feltárása és a regressziós tesztek sikeresek voltak.

Jövőbeni továbbfejlesztési javaslatok (következő verziókhoz):

11. **QR kód alapú rendelésátvétel** - A jelenlegi 4 jegyű azonosító mellett a rendszer generálhatna egy egyedi QR-kódot is a nyugtára.
12. **Paypal / SimplePay** valós bankkártyás fizetés integrálása
13. **Heti akciók és dinamikus árazás -** Az admin panelen előre időzíthető, automatikusan aktiválódó kampányok